Lecture 3: Linear Classifiers

Part 1: Scores, viewpoints, and SVM loss

Last time: Image Classification

Input: image



This image by Nikita is licensed under CC-BY 2.0

Output: Assign image to one of a fixed set of categories

cat

bird

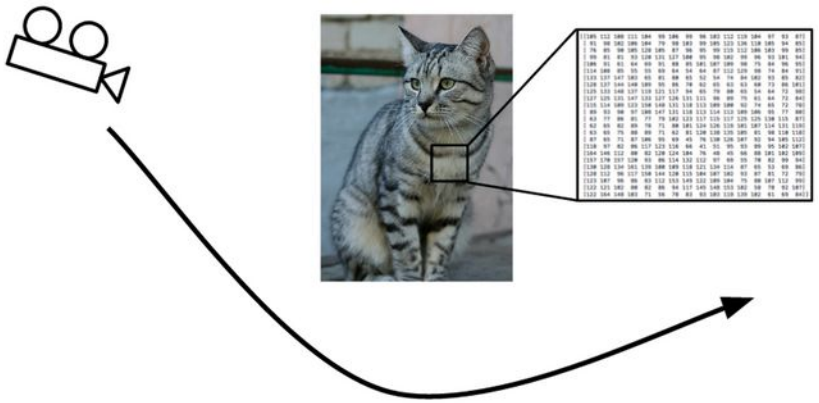
deer

dog

truck

Last Time: Challenges of Recognition

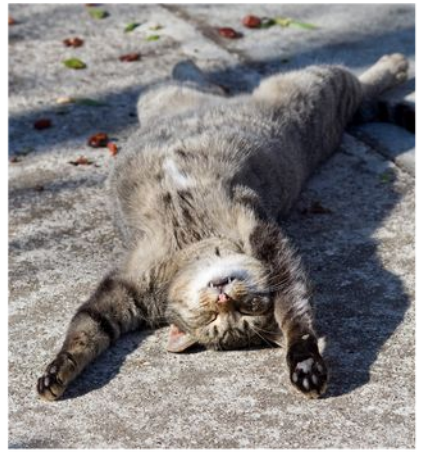
Viewpoint



Illumination



Deformation



Occlusion



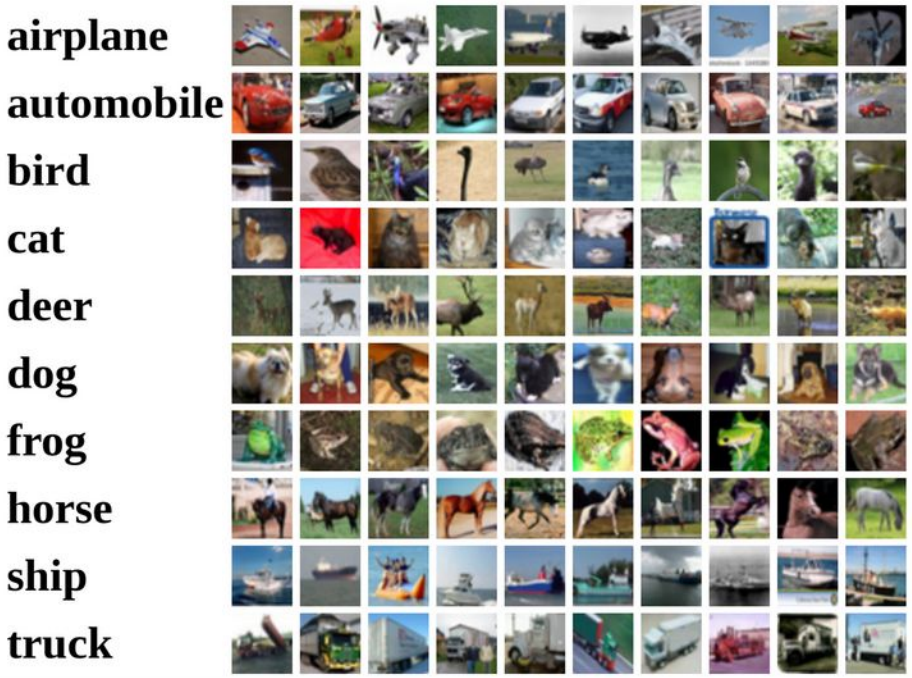
Clutter



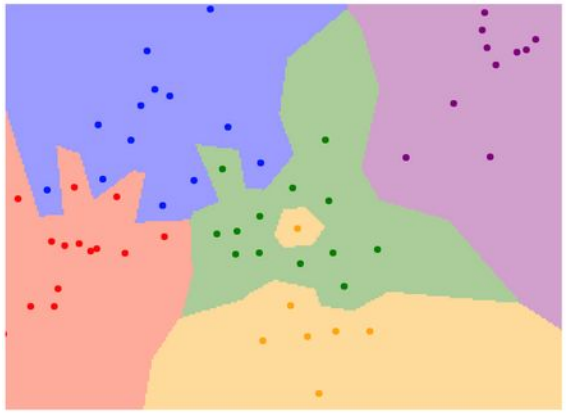
Intraclass Variation



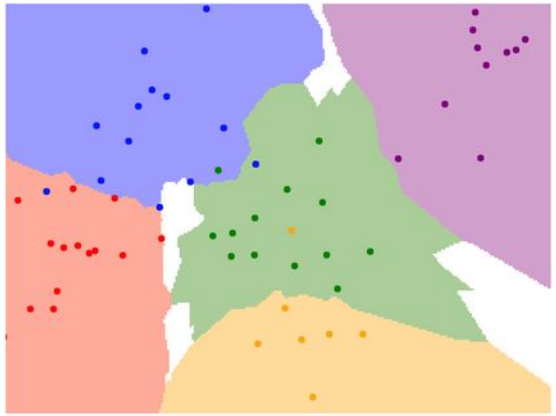
Last time: Data-Driven Approach, kNN

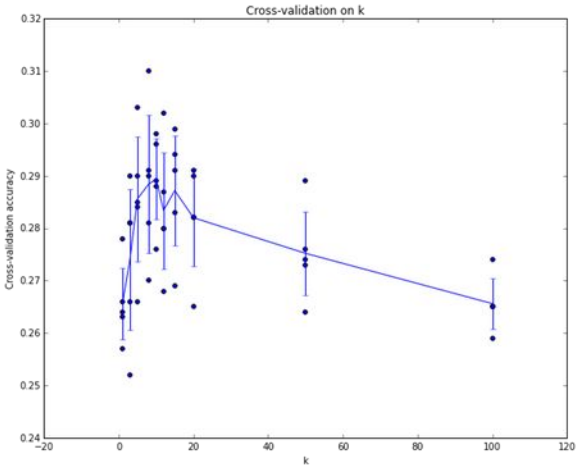


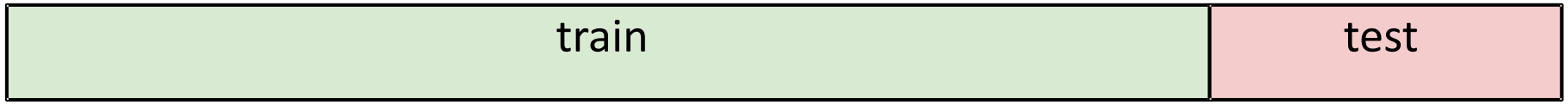
1-NN classifier

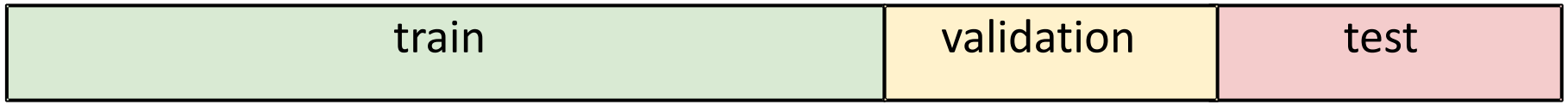


5-NN classifier









Today: Linear Classifiers

Neural Network

Linear classifiers



This image is CC0 1.0 public domain

Recall CIFAR10



50,000 training images

each image is **32 × 32 × 3**

10,000 test images.

Parametric Approach

Image



Array of **32 × 32 × 3** numbers (3072 numbers total)

f(**x**,**W**)

W

parameters or weights

10 numbers giving class scores

Parametric Approach: Linear Classifier

$$f(x,W) = Wx$$

Image



Array of **32 × 32 × 3** numbers (3072 numbers total)

f(**x**,**W**)

W

parameters or weights

10 numbers giving class scores

Parametric Approach: Linear Classifier

$$f(x,W) = Wx$$

(3072,)

(10,)

(10, 3072)

Image



Array of **32 × 32 × 3** numbers (3072 numbers total)

f(**x**,**W**)

10 numbers giving class scores

W

parameters or weights

Parametric Approach: Linear Classifier

$$f(x, W) = Wx + b$$

(10,) (10, 3072) (3072,) (10,)

Image



Array of **32 × 32 × 3** numbers (3072 numbers total)

f(**x**, **W**)

W

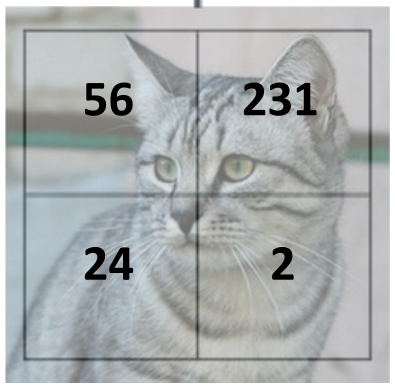
parameters or weights

10 numbers giving class scores

Example for 2 × 2 image, 3 classes (cat/dog/ship)

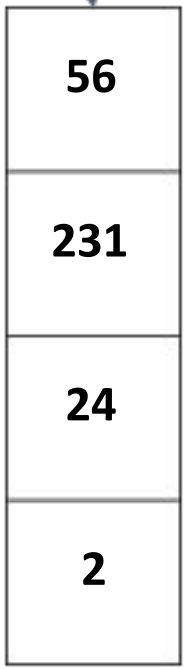
Stretch pixels into column

$f(x,W) = Wx + b$



Input image

(2, 2)

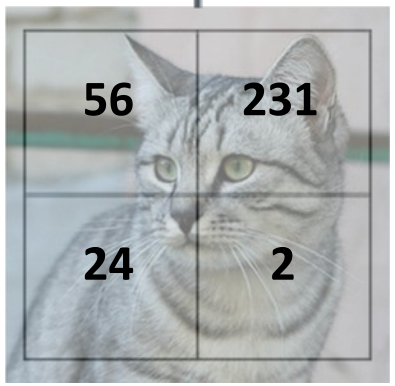


(4,)

Example for 2 × 2 image, 3 classes (cat/dog/ship)

Stretch pixels into column

$f(x,W) = Wx + b$



Input image

(2, 2)

0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.2	-0.3

W (3, 4)

56
231
24
2

(4,)

+

1.1
3.2
-1.2

b

(3,)

=

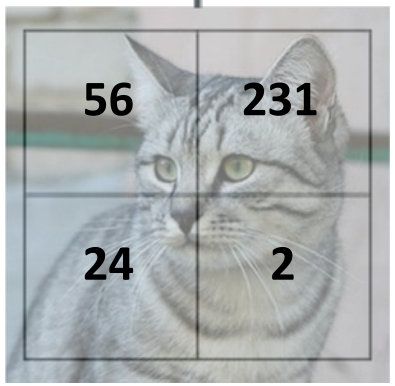
-96.8
437.9
61.95

(3,)

Linear Classifier: Algebraic Viewpoint

Stretch pixels into column

$$f(x,W) = Wx + b$$



Input image

(2, 2)

0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.2	-0.3

W (3, 4)

56
231
24
2

(4,)

+

1.1
3.2
-1.2

b

(3,)

=

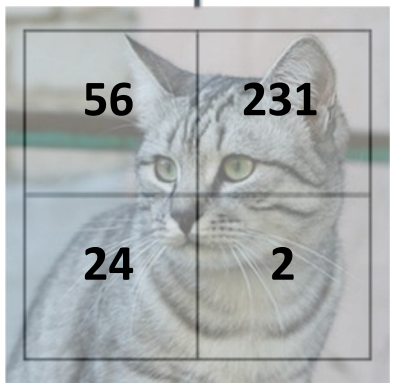
-96.8
437.9
61.95

(3,)

Linear Classifier: Bias Trick

Add extra one to data vector; bias is absorbed into last column of weight matrix

Stretch pixels into column



Input image

(2, 2)

0.2	-0.5	0.1	2.0	1.1
1.5	1.3	2.1	0.0	3.2
0	0.25	0.2	-0.3	-1.2

W (3, 5)

56
231
24
2
1

(5,)

=

-96.8
437.9
61.95

(3,)

Linear Classifier: Predictions are Linear!

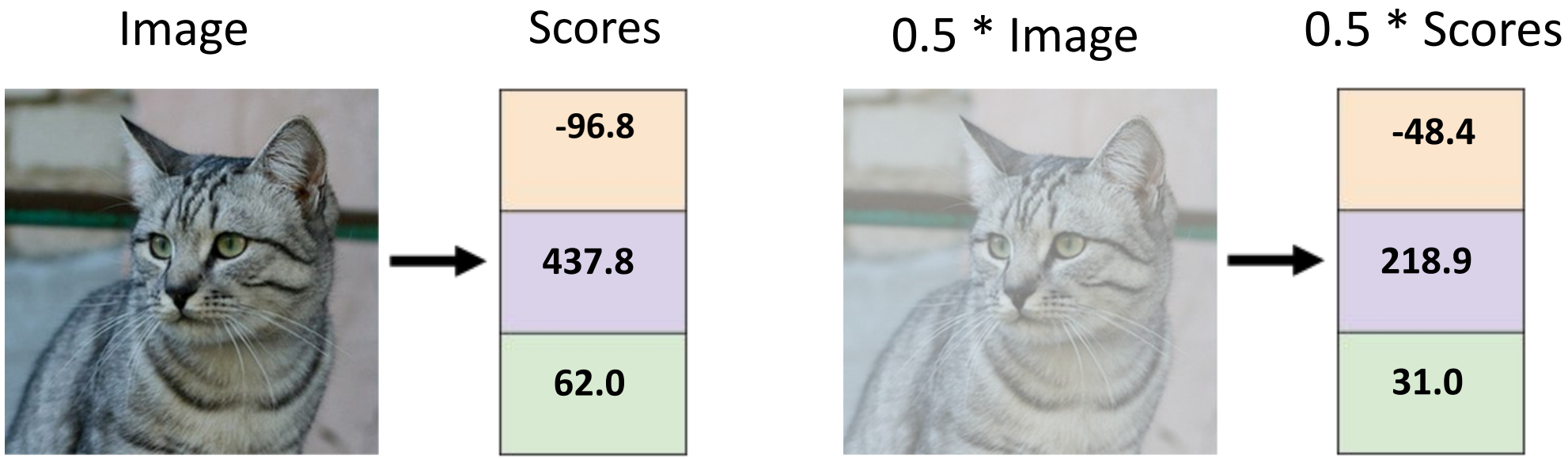
$$f(x,W) = Wx \qquad \text{(ignore bias)}$$

$$f(cx, W) = W(cx) = c * f(x, W)$$

Linear Classifier: Predictions are Linear!

$$f(x,W) = Wx \qquad \text{(ignore bias)}$$

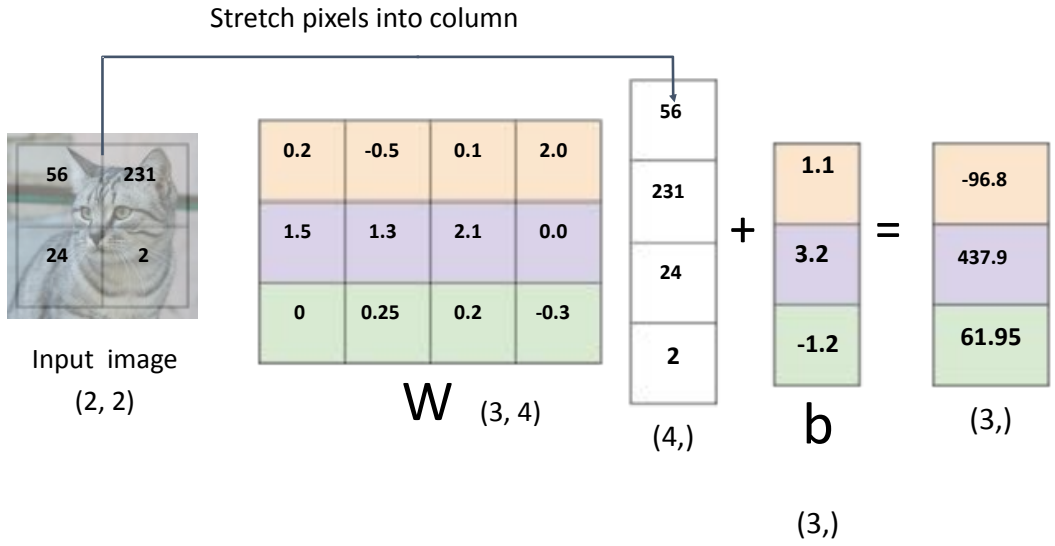
$$f(cx, W) = W(cx) = c * f(x, W)$$



Interpreting a Linear Classifier

Algebraic Viewpoint

$$f(x,W) = Wx + b$$

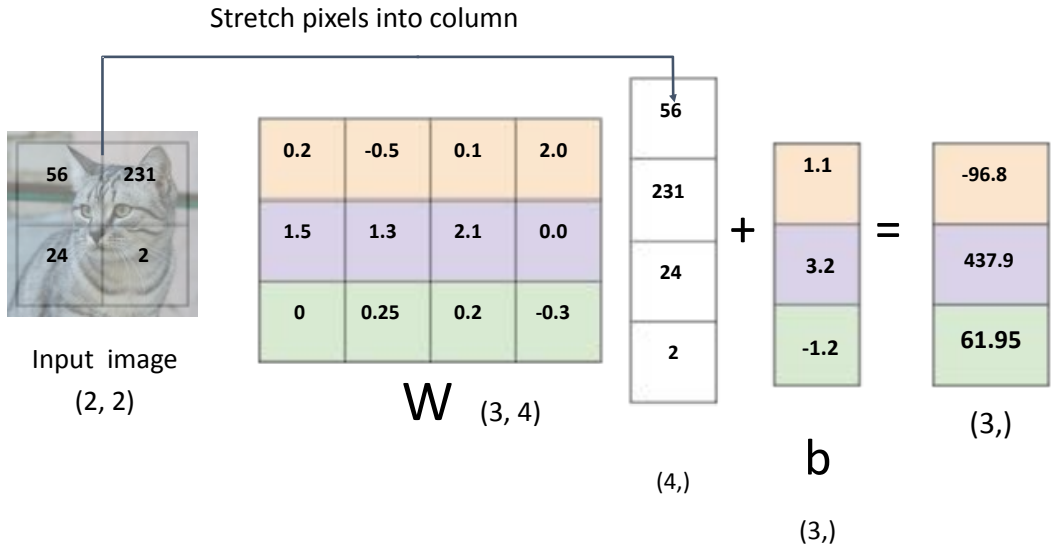


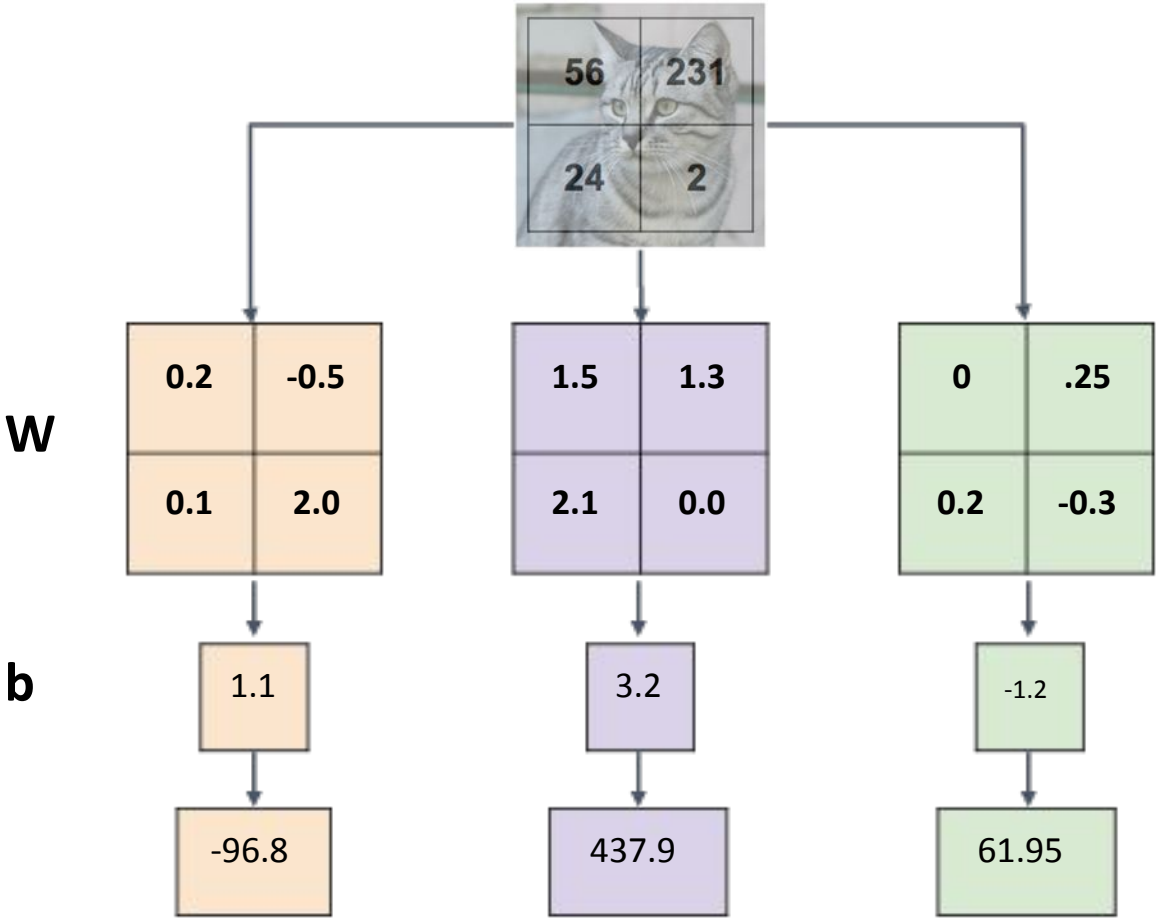
Interpreting a Linear Classifier

Instead of stretching pixels into columns, we can equivalently stretch rows of W into images!

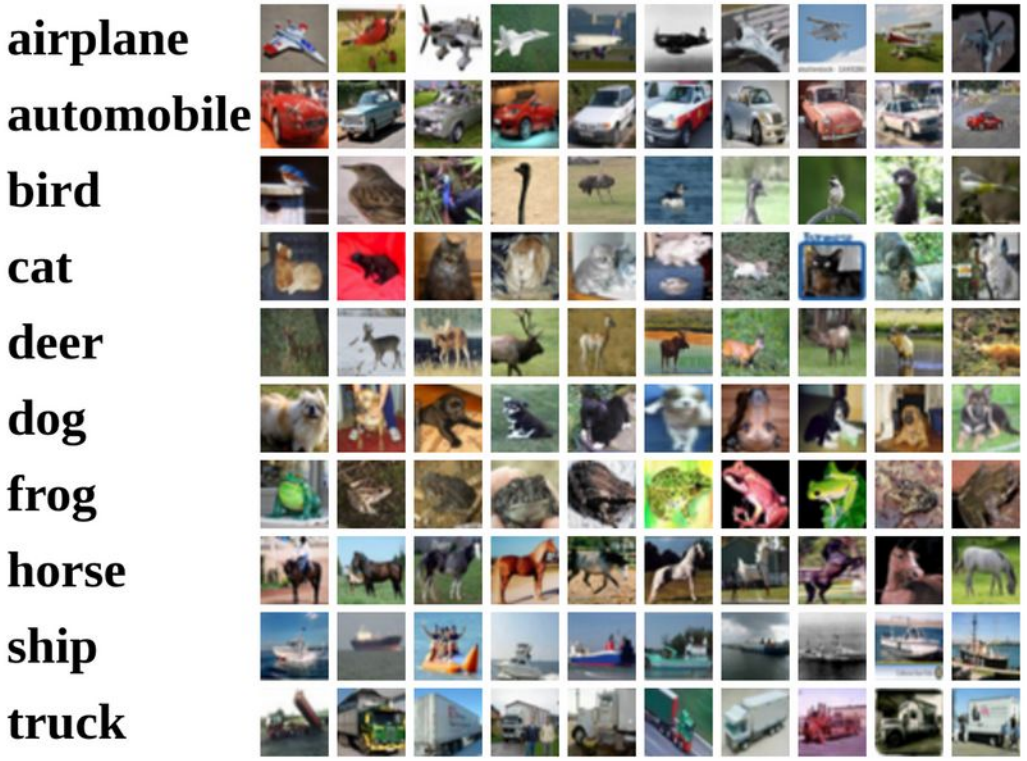
Algebraic Viewpoint

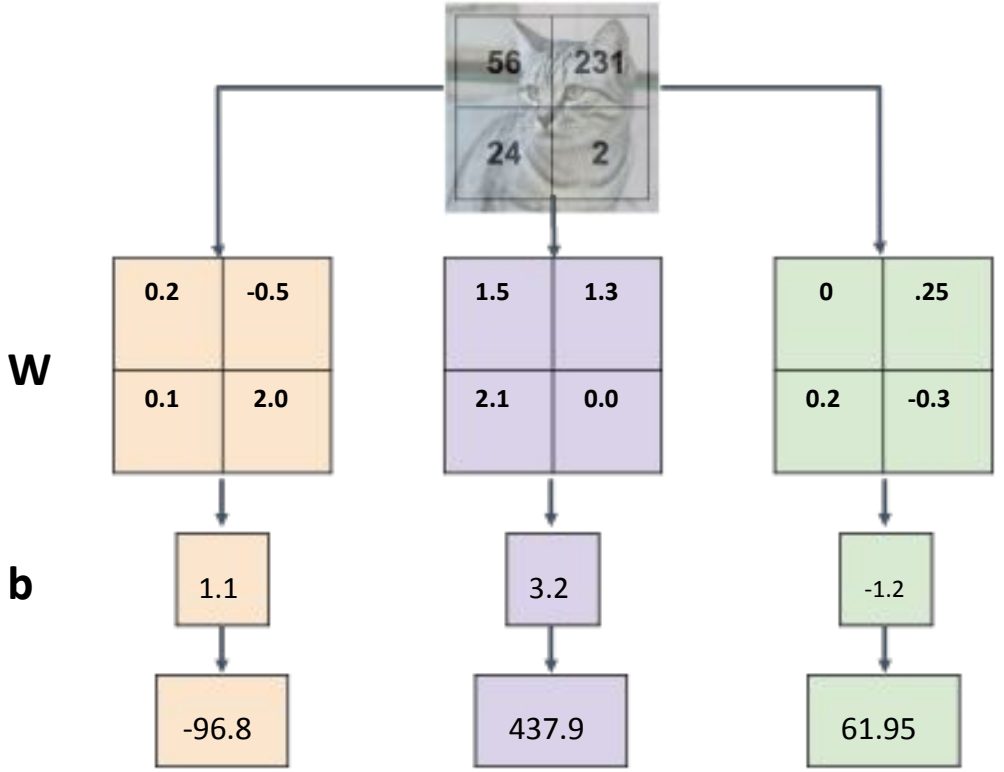
$$f(x,W) = Wx + b$$



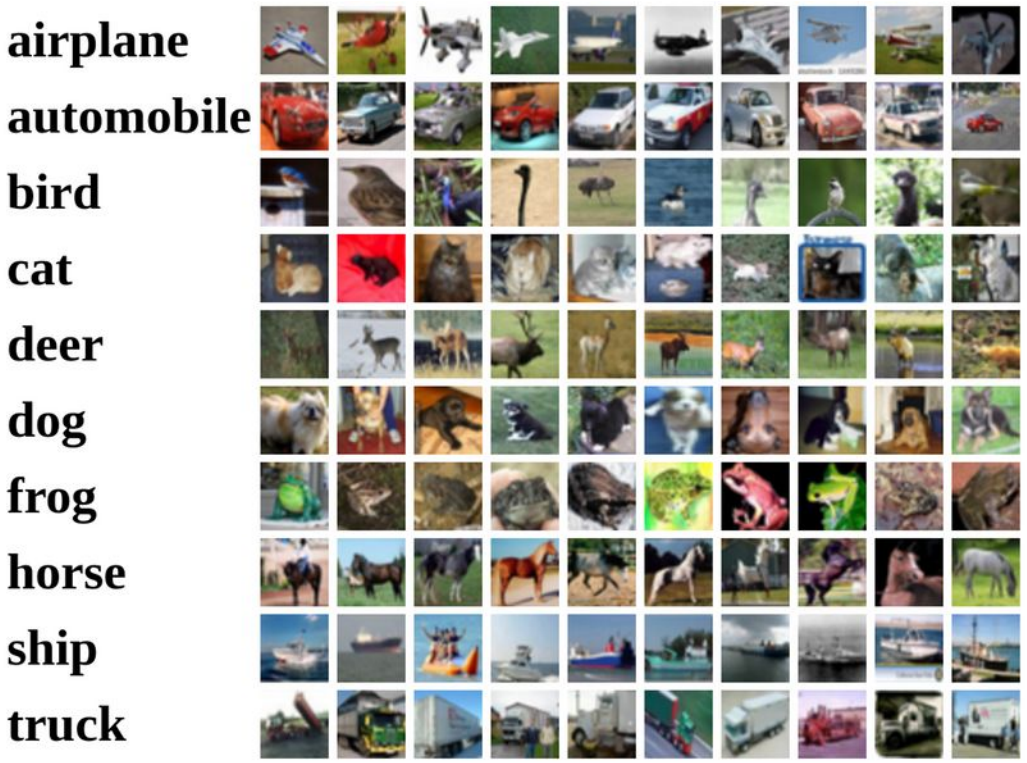


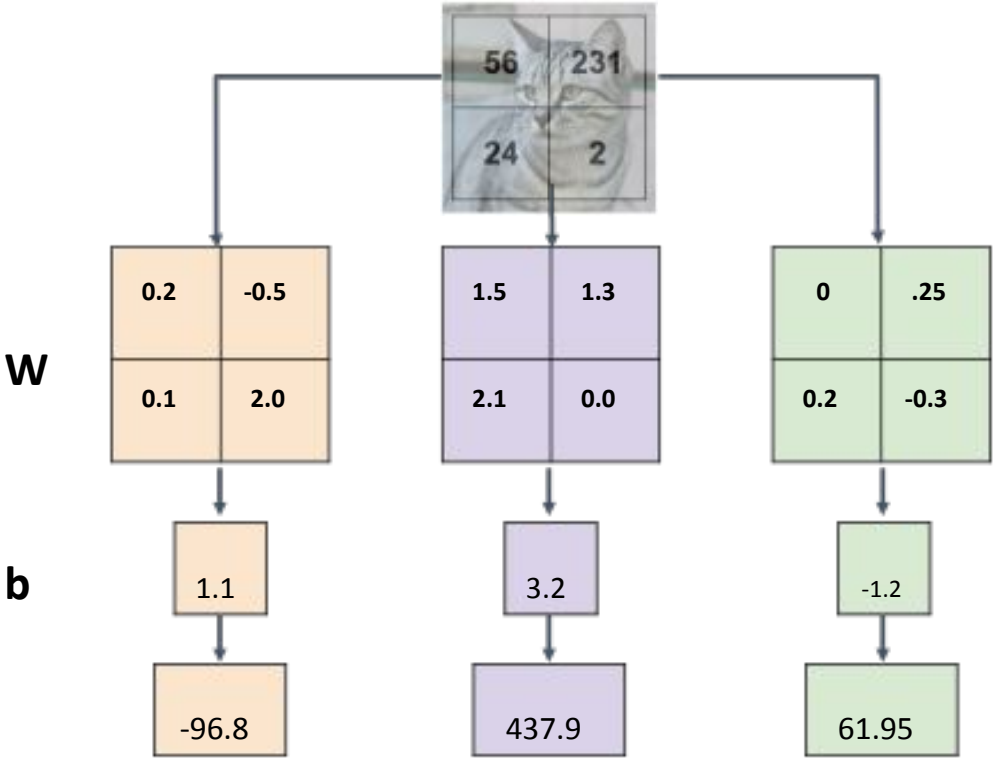
Interpreting a Linear Classifier

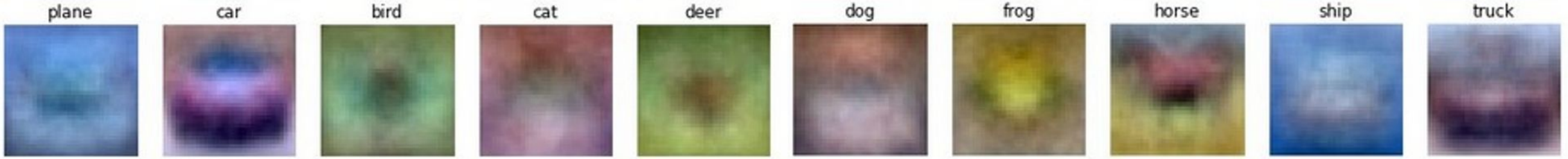




Interpreting a Linear Classifier: Visual Viewpoint

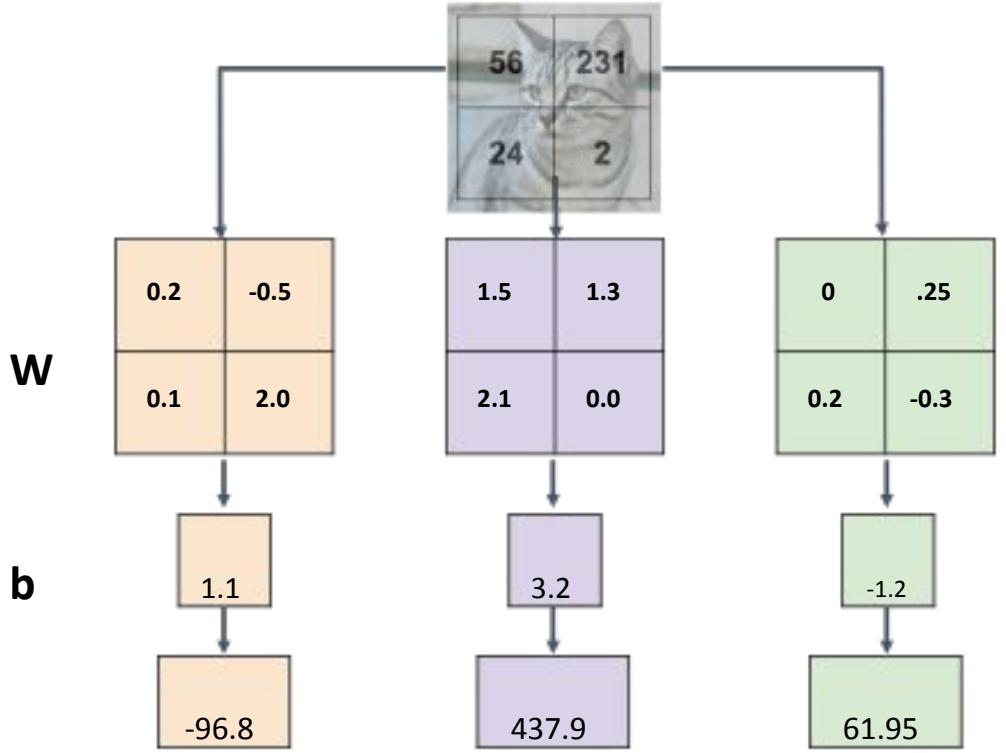






Interpreting a Linear Classifier: Visual Viewpoint

Linear classifier has one "template" per category

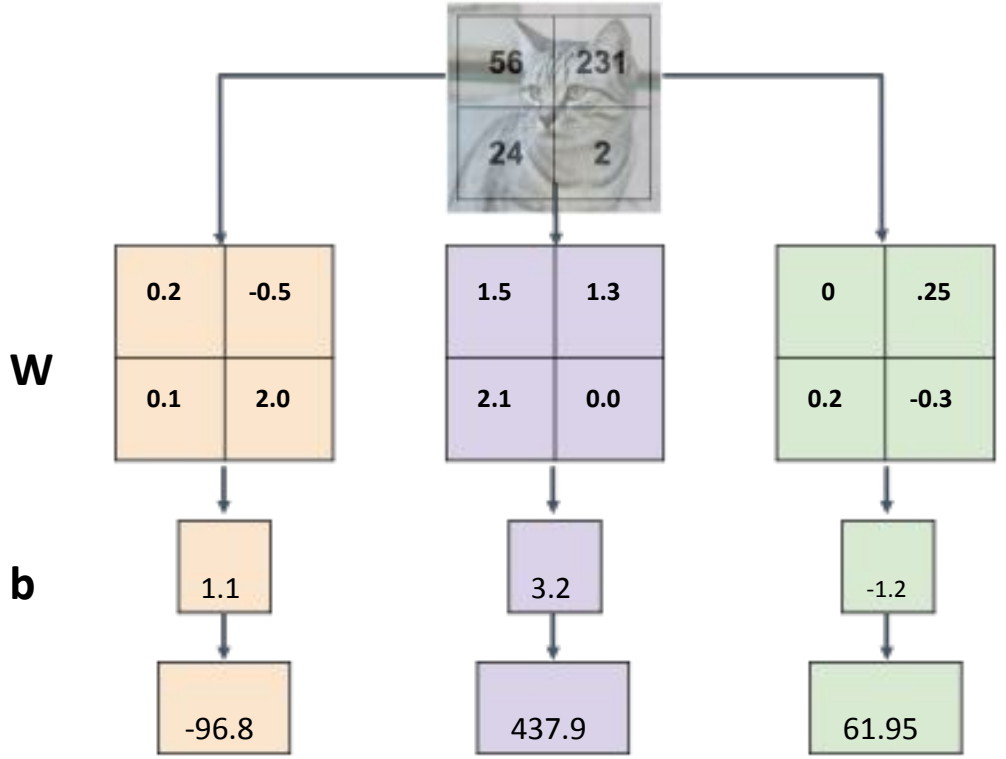




Interpreting a Linear Classifier: Visual Viewpoint

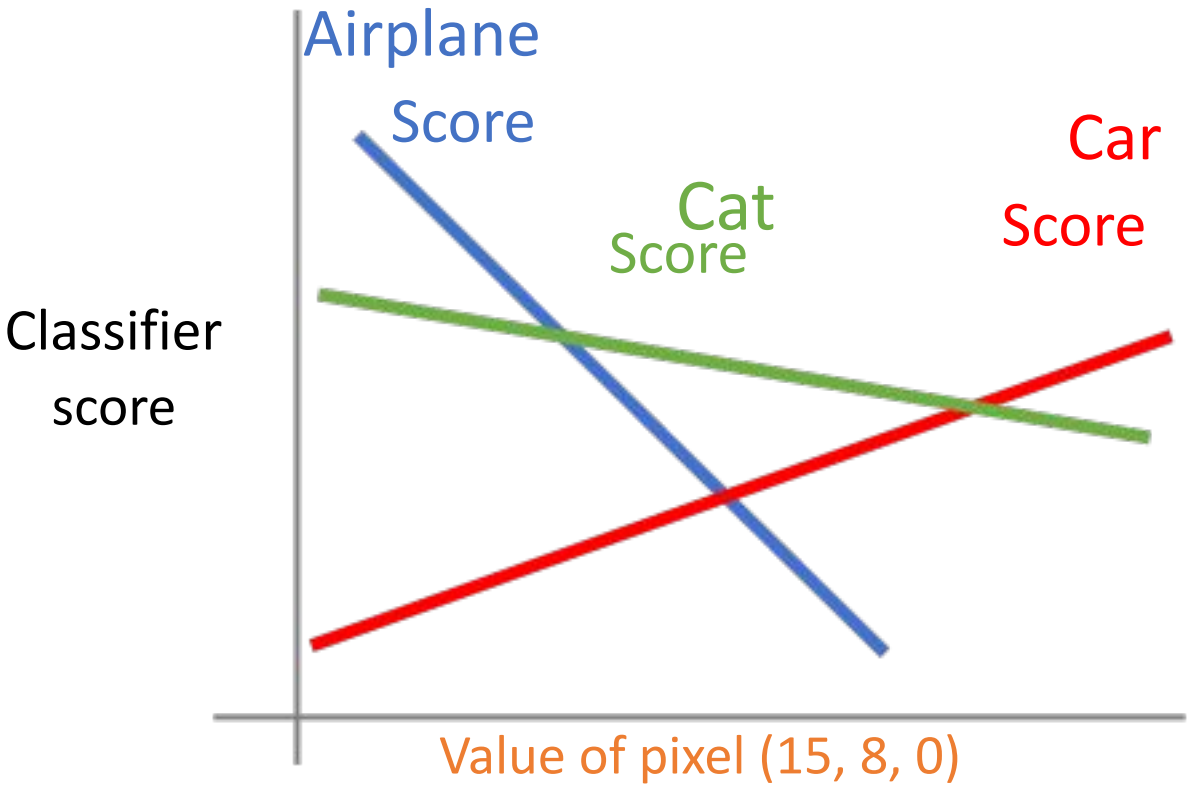
A linear classifier learns one template for each category.

A single template cannot represent every appearance of a class. For example, the learned horse template may contain two heads.





Interpreting a Linear Classifier: Geometric Viewpoint

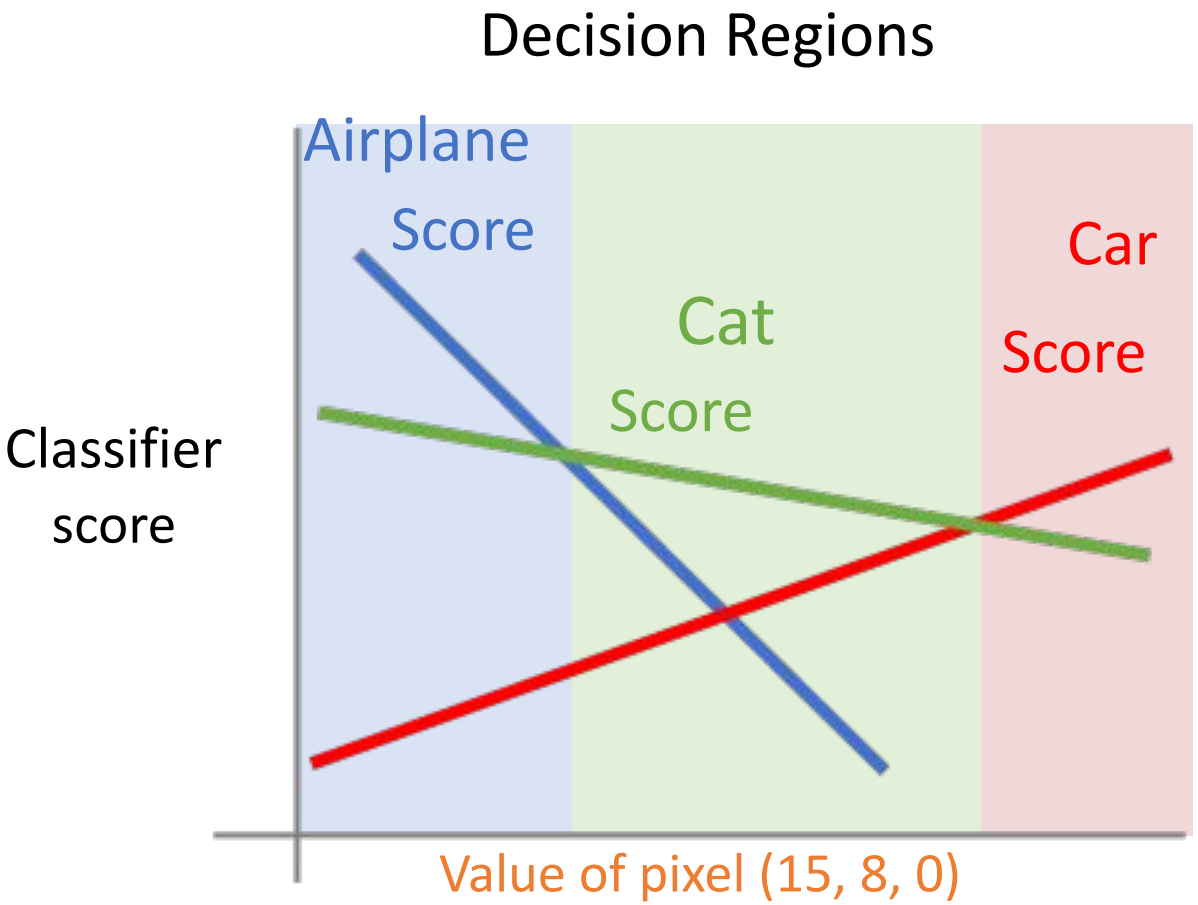


$$f(x,W) = Wx + b$$



Array of **32 × 32 × 3** numbers (3072 numbers total)

Interpreting a Linear Classifier: Geometric Viewpoint

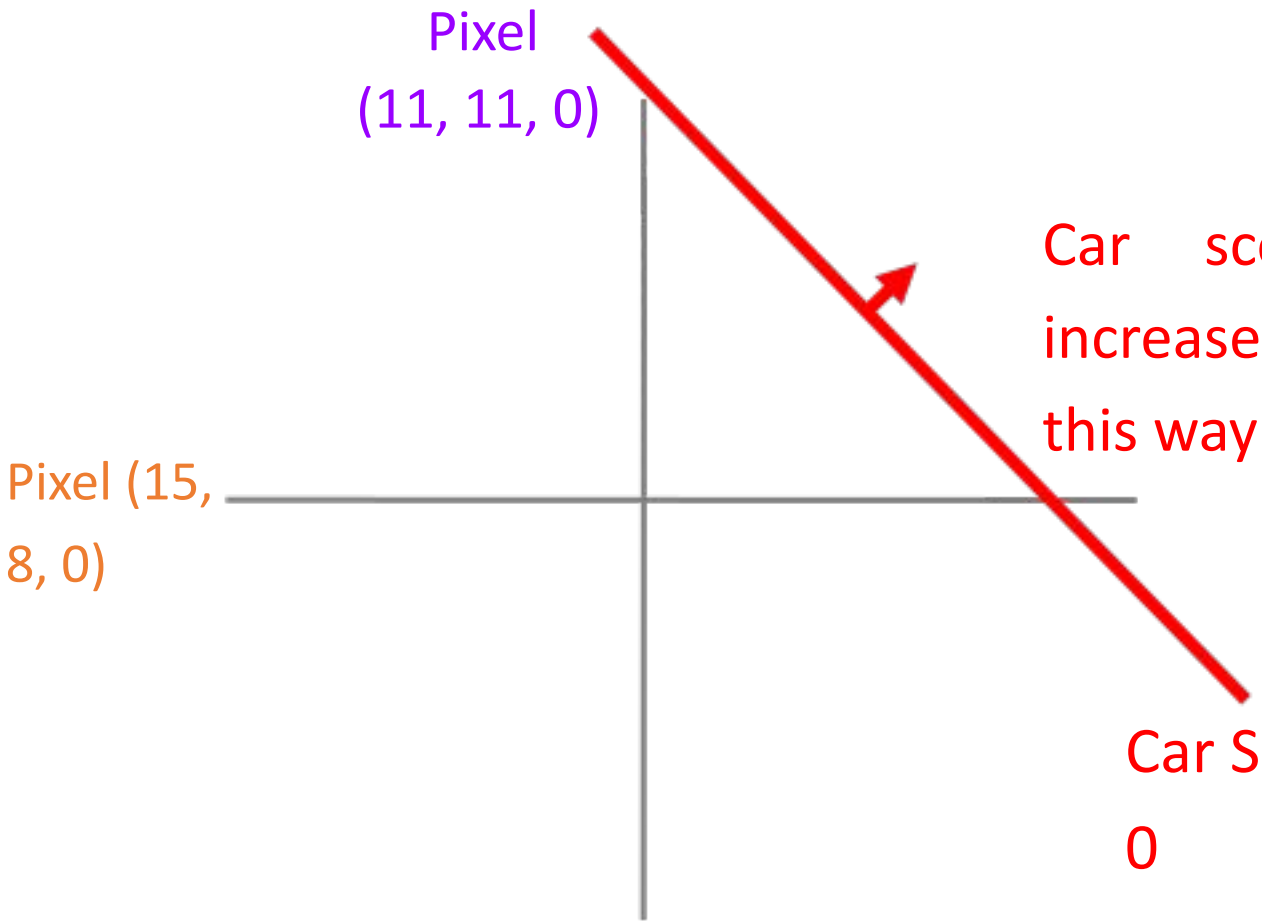


$$f(x,W) = Wx + b$$

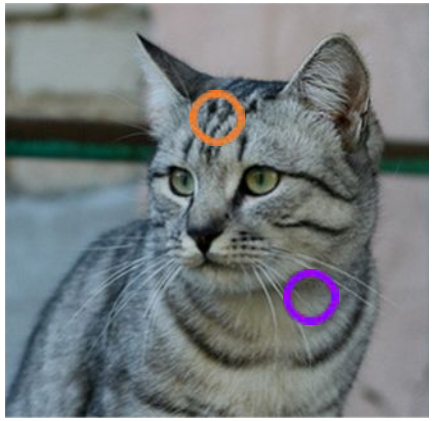


Array of **32 × 32 × 3** numbers (3072 numbers total)

Interpreting a Linear Classifier: Geometric Viewpoint

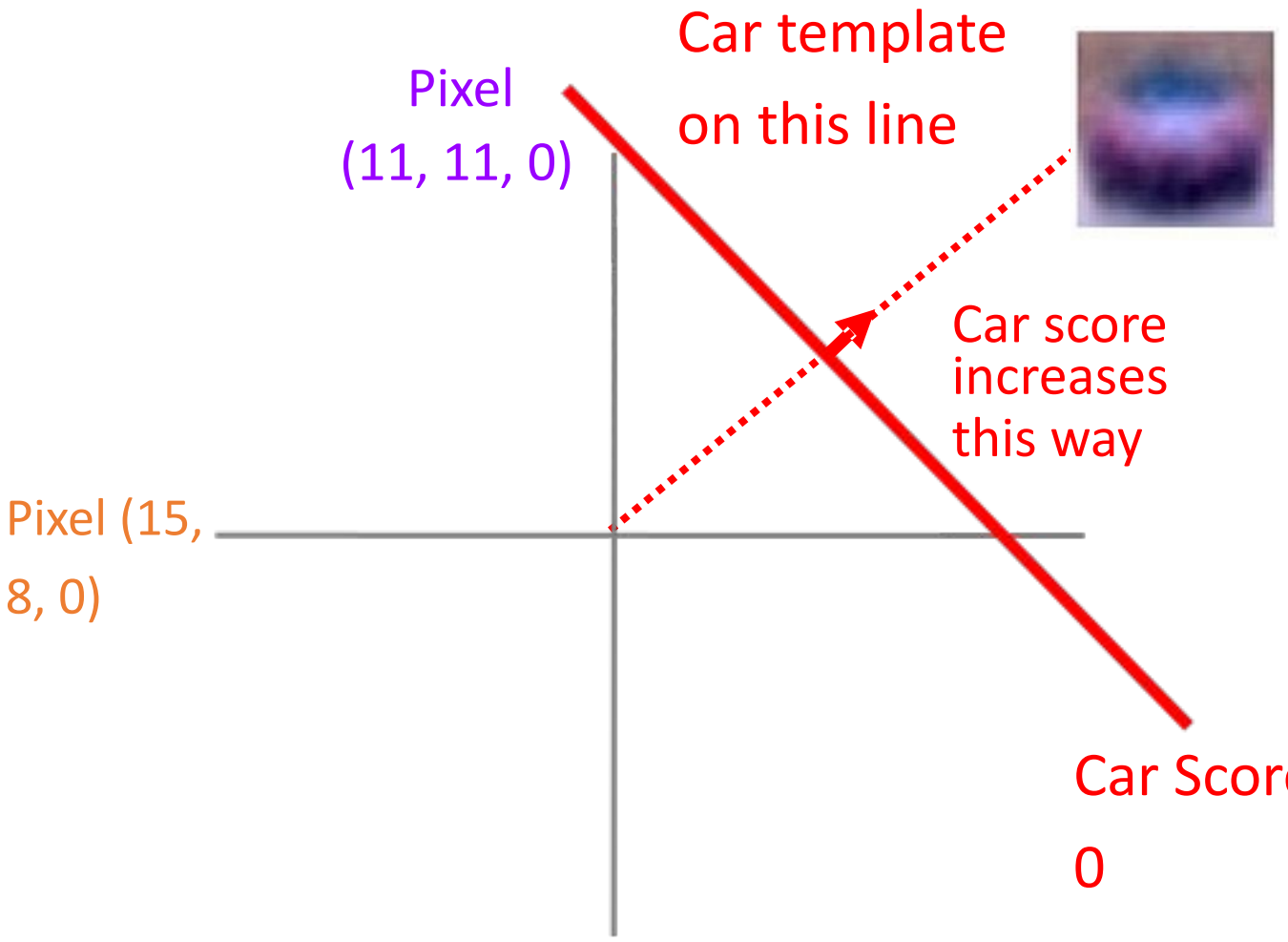


$$f(x,W) = Wx + b$$

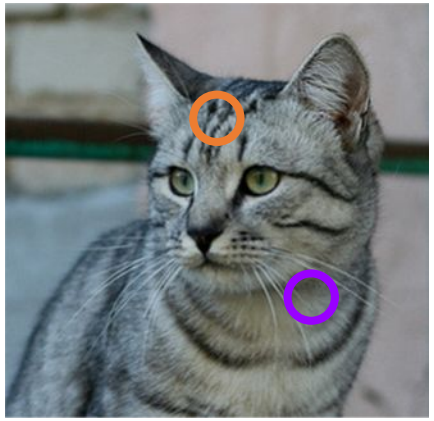


Array of **32 × 32 × 3** numbers (3072 numbers total)

Interpreting a Linear Classifier: Geometric Viewpoint

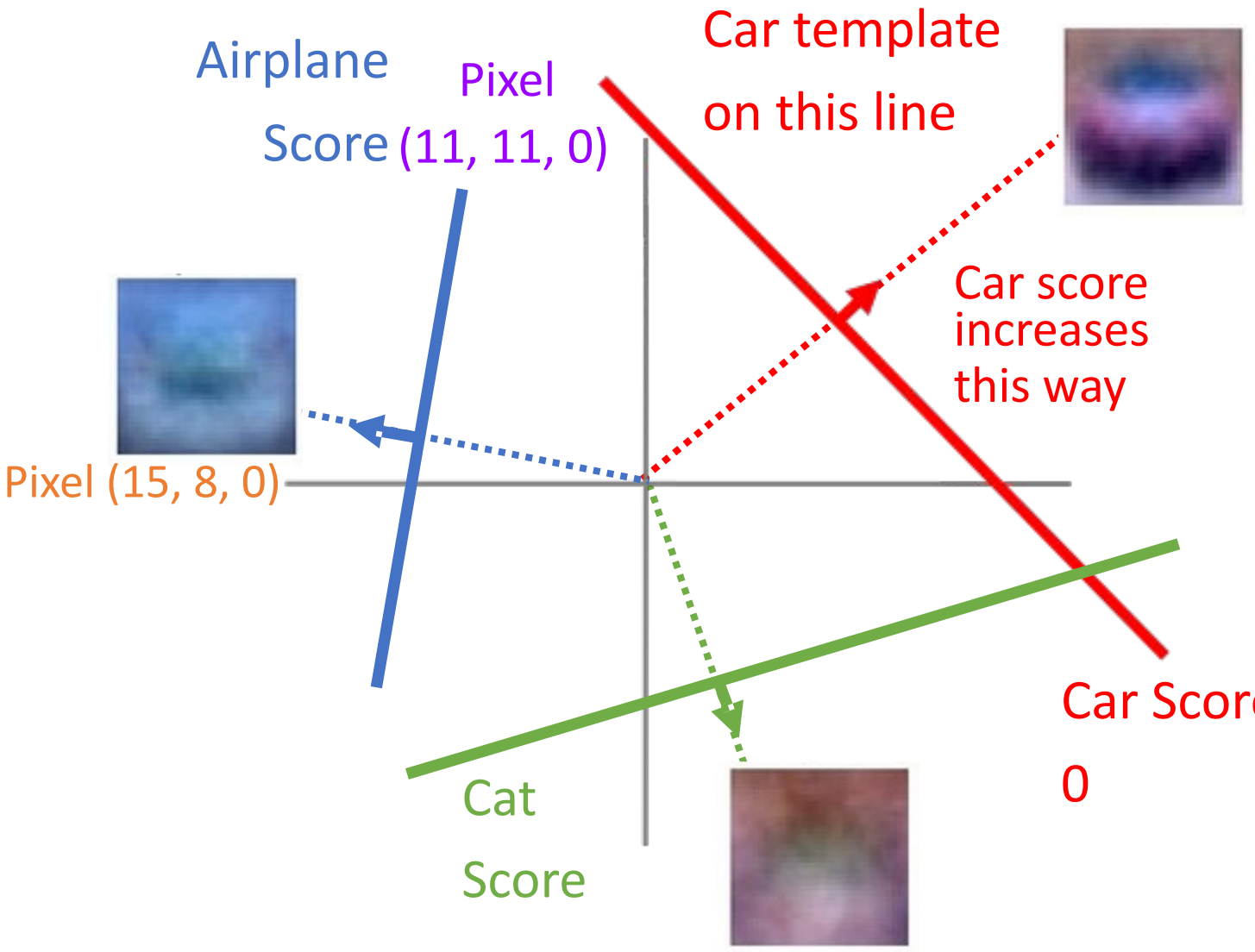


$$f(x,W) = Wx + b$$

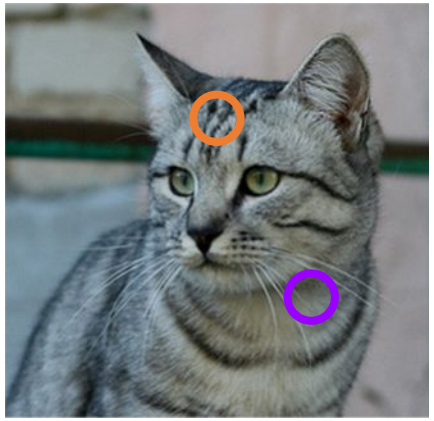


Array of **32 × 32 × 3** numbers (3072 numbers total)

Interpreting a Linear Classifier: Geometric Viewpoint

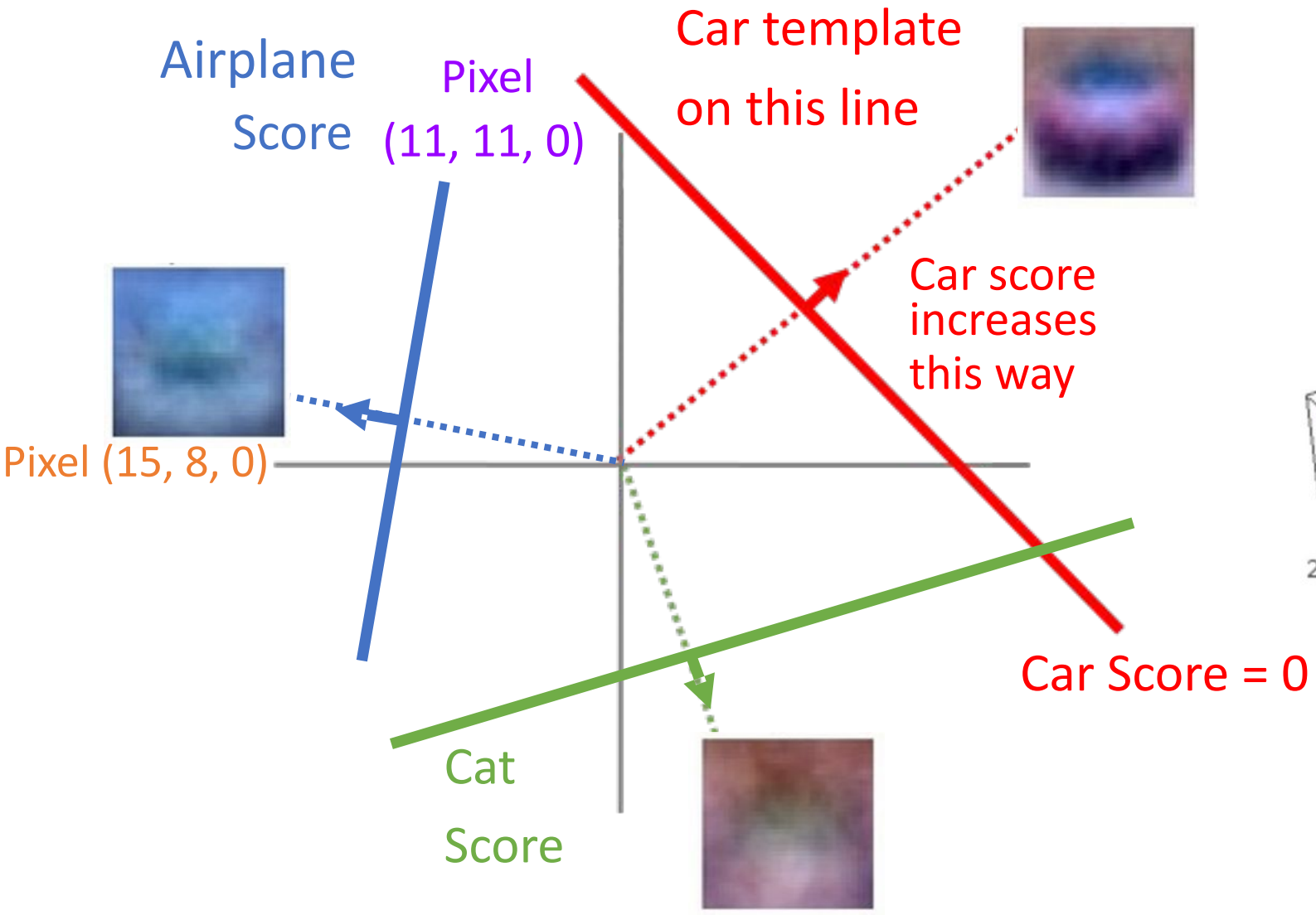


$$f(x,W) = Wx + b$$

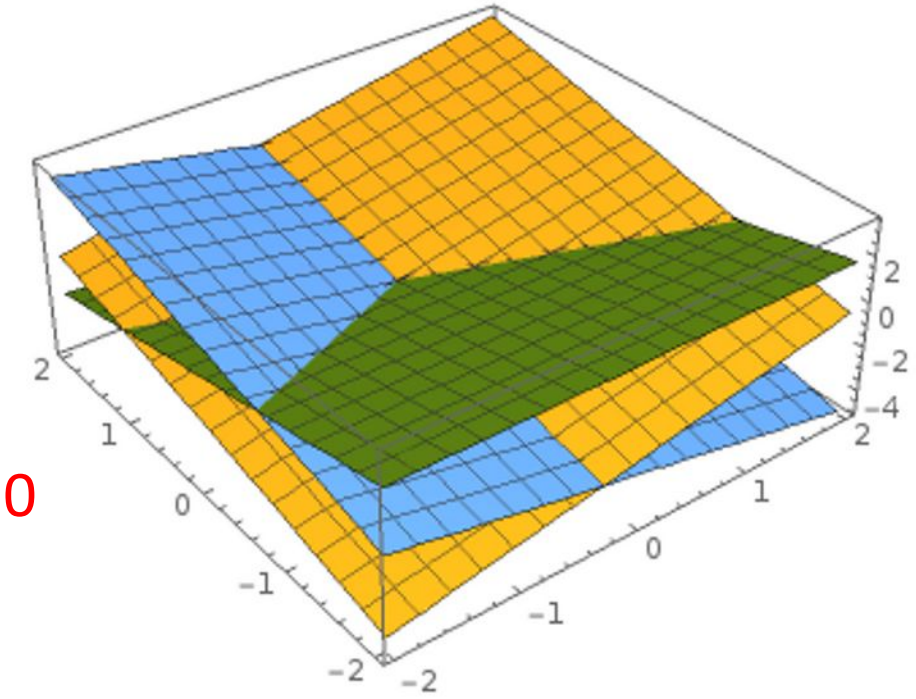


Array of **32 × 32 × 3** numbers (3072 numbers total)

Interpreting a Linear Classifier: Geometric Viewpoint



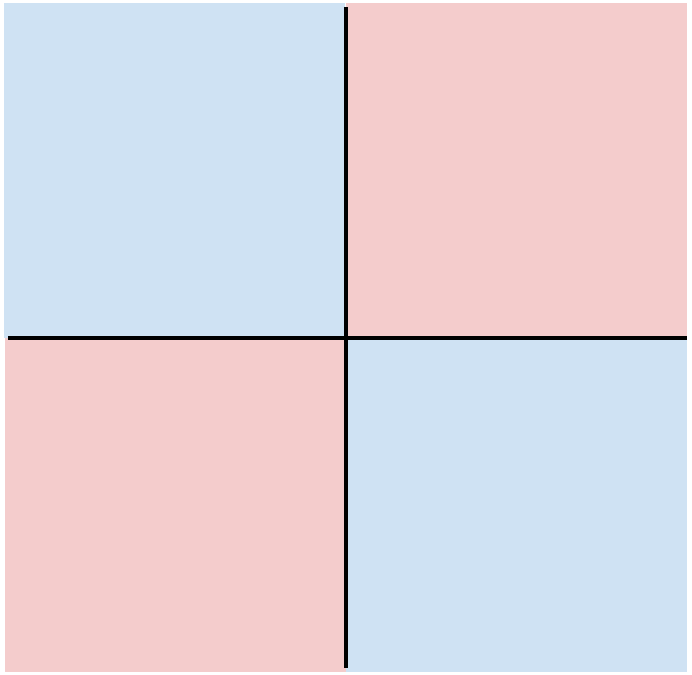
Hyperplanes carving up a high-dimensional space



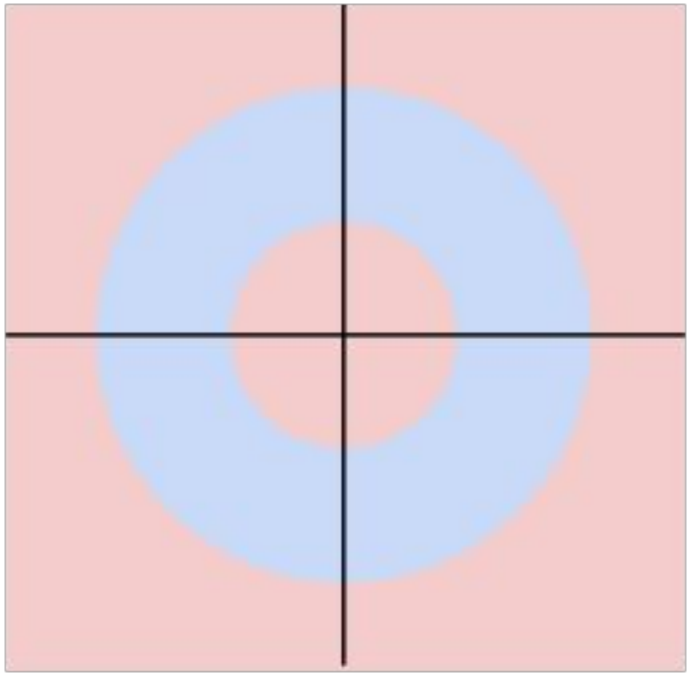
Plot created using Wolfram Cloud

Hard Cases for a Linear Classifier

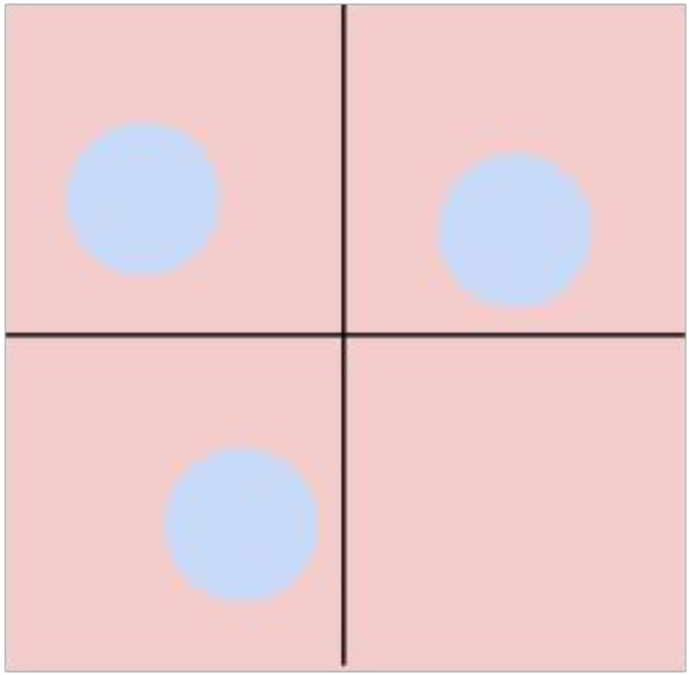
Class 1: first and third quadrants
Class 2: second and fourth quadrants



Class 1: 1 ≤ L2 norm ≤ 2
Class 2: everything else

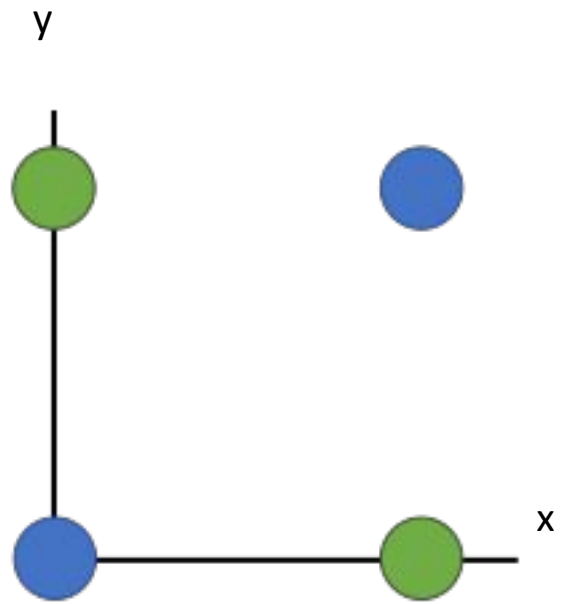


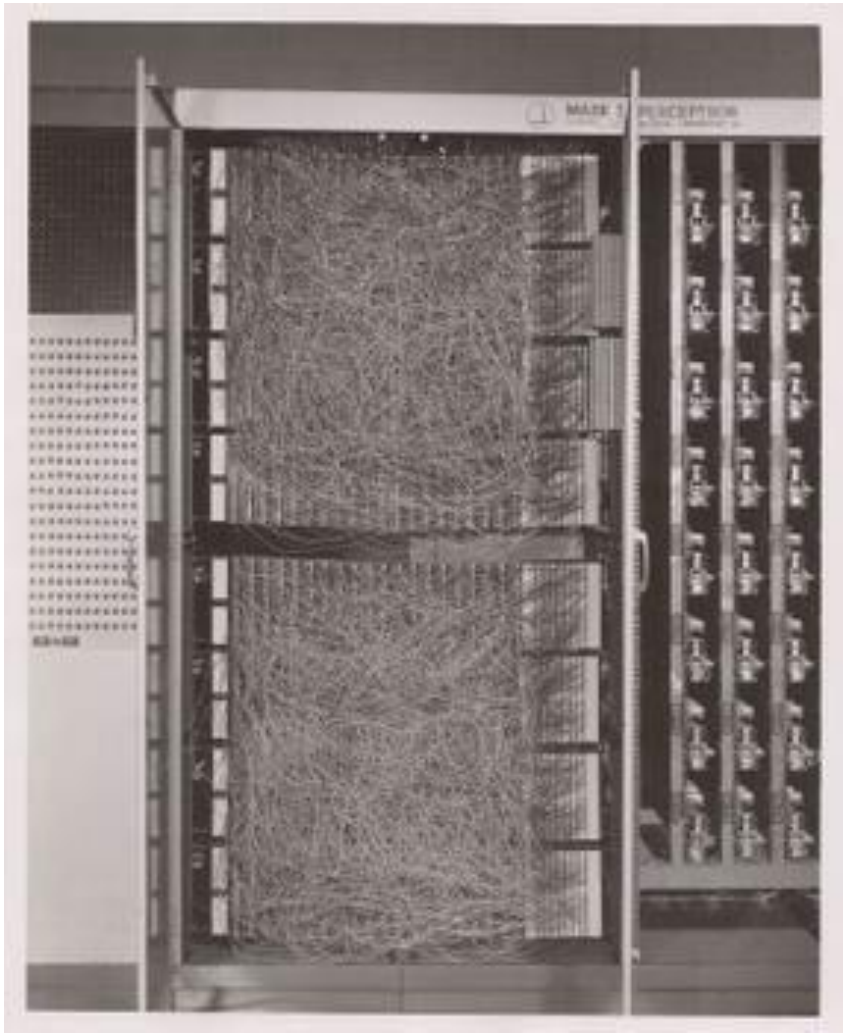
Class 1: three separate modes
Class 2: everything else



Recall: Perceptron couldn’t learn XOR

X	Y	F(x,y)
0	0	0
0	1	1
1	0	1
1	1	0

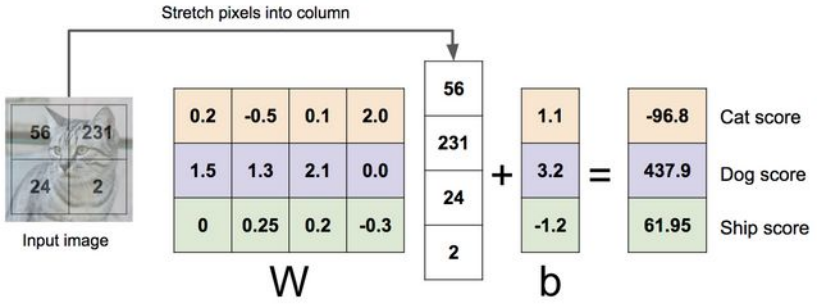




Linear Classifier: Three Viewpoints

Algebraic Viewpoint

f(x,W) = Wx



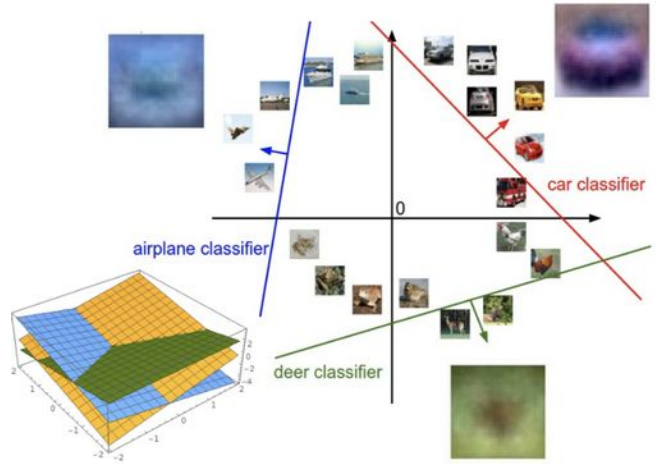
Visual Viewpoint

One template
per class



Geometric Viewpoint

Hyperplanes
cutting up space



A Linear Score Function

$$f(x, W) = Wx + b$$





airplane	-3.45	-0.51	3.42
automobile	-8.87	**6.04**	4.64
bird	0.09	5.31	2.65
cat	**2.9**	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	**-4.34**
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

Given W, the score function computes one score for each class.

Training finds a value of W that produces useful scores.

Cat image by Nikita is licensed under CC-BY 2.0; Car image is CC0 1.0 public domain; Frog image is in the public domain

Choosing a good W

$f(x,W) = Wx + b$





airplane	-3.45	-0.51	3.42
automobile	-8.87	**6.04**	4.64
bird	0.09	5.31	2.65
cat	**2.9**	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	**-4.34**
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

1. Use a loss function to measure the quality of W.

2. Find the value of W that minimizes the loss.

This search process is called optimization.

Loss Function

A **loss function** tells how good our current classifier is

Low loss = good classifier
High loss = bad classifier

(Also called: **objective function**; **cost function**)

Negative loss function sometimes called **reward function**, **profit function**, **utility function**, **fitness function**, etc

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^{N}$$

Where x_i is image and
y_i is (integer) label

Loss Function

A **loss function** tells how good our current classifier is

Low loss = good classifier
High loss = bad classifier

(Also called: **objective function**; **cost function**)

Negative loss function sometimes called **reward function**, **profit function**, **utility function**, **fitness function**, etc

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^{N}$$

Where x_i is image and
y_i is (integer) label

Loss for a single example is

$$L_i(f(x_i, W), y_i)$$

Loss Function

A **loss function** tells how good our current classifier is

Low loss = good classifier
High loss = bad classifier

(Also called: **objective function**; **cost function**)

Negative loss function sometimes called **reward function**, **profit function**, **utility function**, **fitness function**, etc

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^{N}$$

Where x_i is image and
y_i is (integer) label

Loss for a single example is

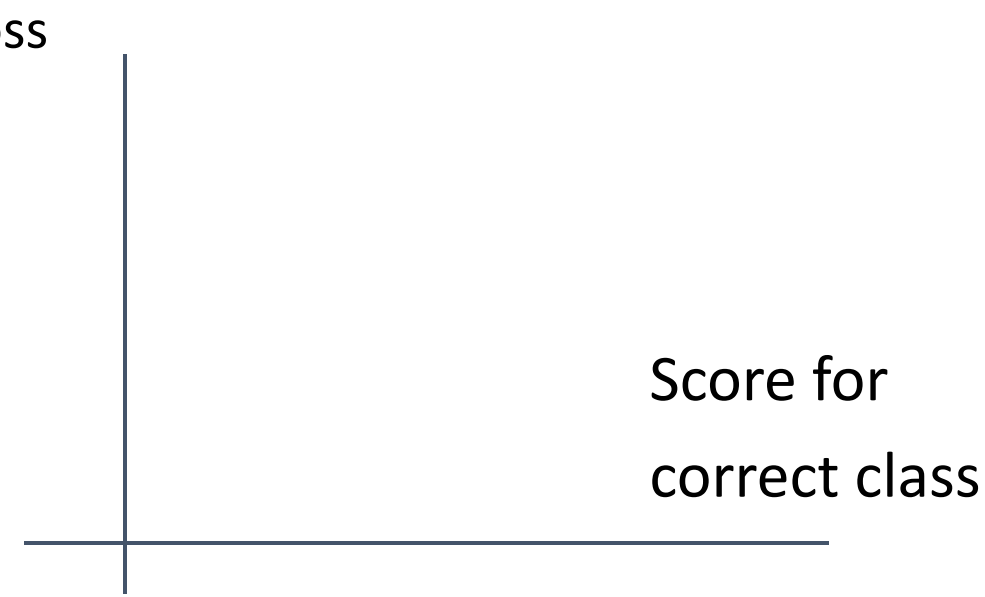
$$L_i(f(x_i, W), y_i)$$

Loss for the dataset is average of per-example losses:

$$L = \frac{1}{N}\sum_i L_i(f(x_i, W), y_i)$$

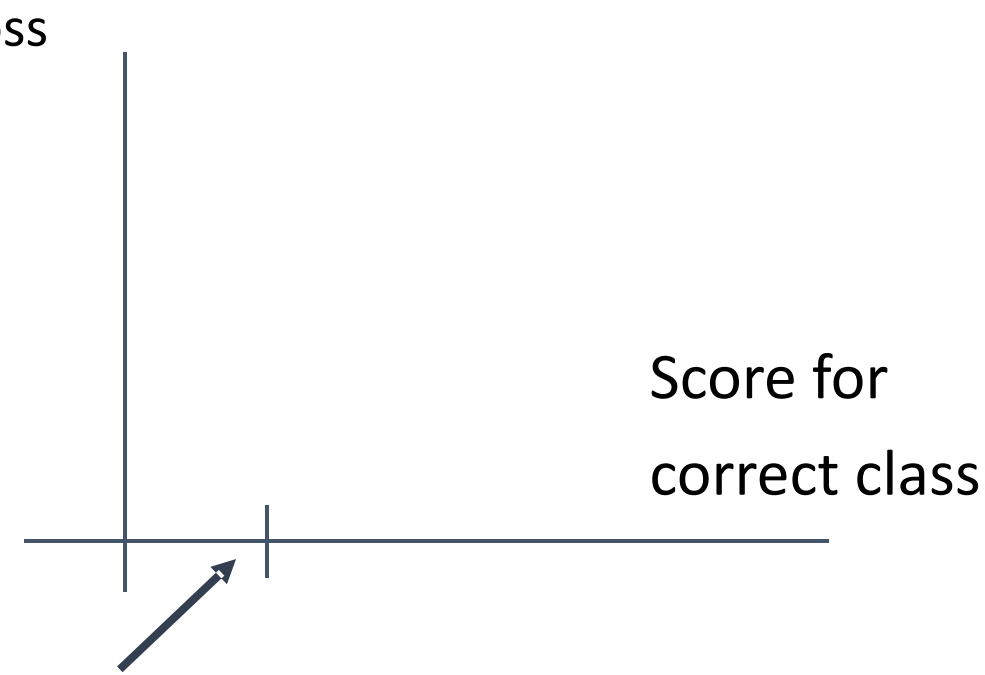
Multiclass SVM Loss

”The score of the correct class should be higher than all the other scores”



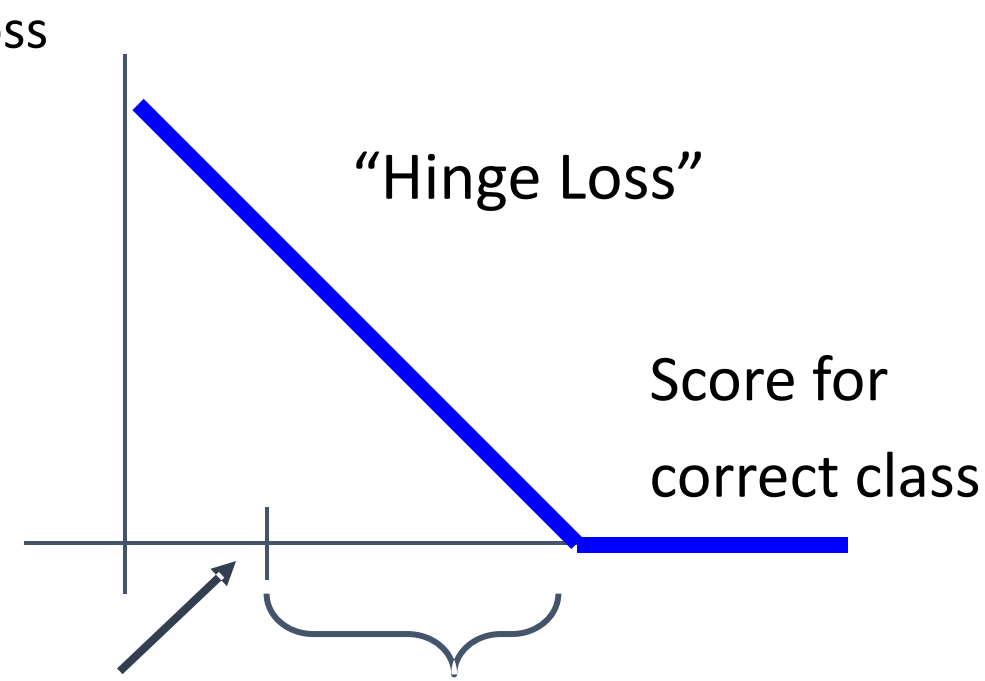
Multiclass SVM Loss

”The score of the correct class should be higher than all the other scores”



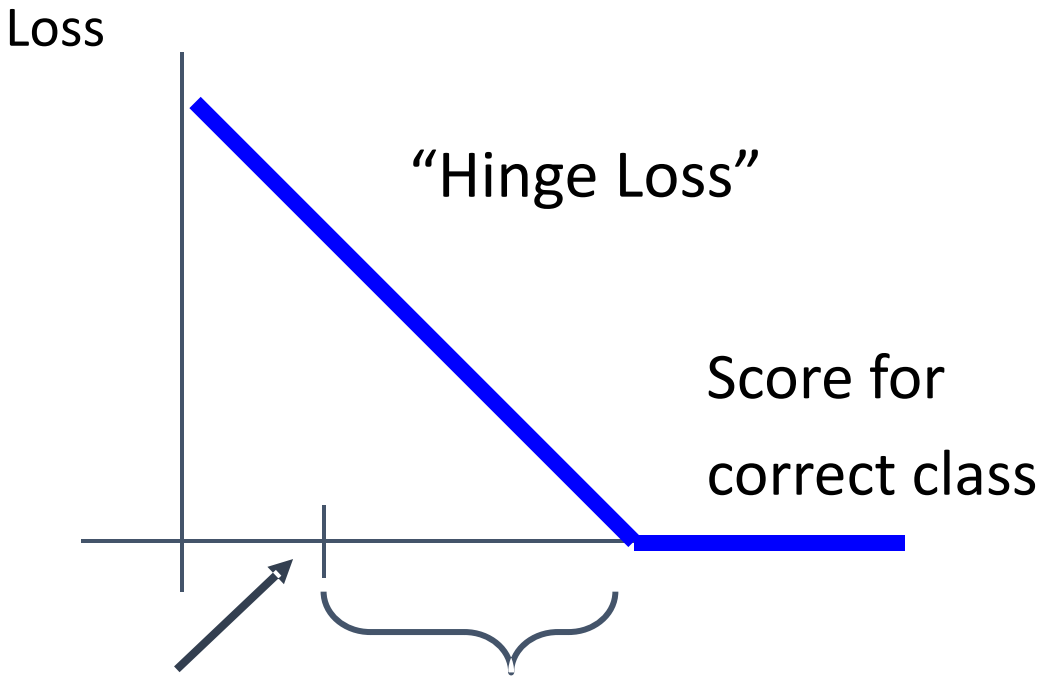
Multiclass SVM Loss

”The score of the correct class should be higher than all the other scores”



Multiclass SVM Loss

"The score of the correct class should be higher than all the other scores"



Given an example (x_i, y_i)
(x_i is image, y_i is label)

Let $s = f(x_i, W)$ be scores

Then the SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Part 1 recap and Lab 04 preparation

Linear score function	The model computes class scores with $f(x, W) = Wx + b$.
Three interpretations	View the classifier algebraically, as visual templates, or as geometric decision boundaries.
Multiclass SVM	The hinge loss rewards the correct class score when it exceeds every incorrect score by the margin.
Lab 04	Implement the naive SVM loss and verify its analytic gradient.